DATE: / /

ASSIGNMENT #02

Name :- Ammar Rashid
SAP ID :- 67006
Subject :- Analysis of Algorithm

Problem 1 :- Quadratic Method

Solution :-

As we know for sum formula

$$S = \frac{N}{2}(2a_1 + (N-1)d)$$

$$2S = N(2a_1 + dN - d)$$

$$2S = dN^2 + 2a_1N - dN$$

$$0 = dN^2 + (2a_1 - d)N - 2S$$

$$dN^2 + (2a_1 - d)N - 2S = 0$$

$$Ax^2 + Bx + C = 0$$

Algorithm :-

Algorithm Quadratic(Sum1, A1, Cd, N)
[Algorithm to find sum and last term of arithmetic using quadratic]

Step 1 :- [Read the sum, first term and common difference]
Read (Sum1, A1, Cd)

Step 2 : [Initialize the coefficients for $Ax^2 + Bx + C = 0$]

DATE: / /

$A = Cd$

$B = 2A1 - Cd$

$C = -2\,Sum1$

Step 4: [Calculate the D]

$D = B^2 - 4AC$

Step 5: [~~Loop~~ Condition to check if the solution is real]

If $(D < 0)$ then

write ("NO real solution")

else

$N1 = (-B + \sqrt{D}) / 2A$

$N2 = (-B - \sqrt{D}) / 2A$

endif

Step 6: [Selecting the positive integer as valid Number of Terms]

If $(N1 > 0)$ then

$N = N1$

Elseif $(N2 > 0)$ then

$N = N2$

else

return false.

endif

Step 7: [Calculate last term y]

$y = A1 + (N - 1)Cd$

DATE: / /

Step 8 : [Display]

Write (N, y)

Step 9 : [Finish]

Exist.

Pseudo Code :-

```
Quadratic (S, a1, d):
    A = d;
    B = 2 * a1 - d;
    C = -2 * S;
    D = B * B - 4 * A * C;

    If (D < 0)
        printf("%d")
    End If
    N1 = (-B + sqrt(D)) / (2 * A);
    N2 = (-B - sqrt(D)) / (2 * A);

    N = 0;
    If (N1 > 0)
        N = N1
    Elseif (N2 > 0)
        N = N2
    Else
        printf("%d")
    End If
```

DATE: / /

```
y = a1 + (N-1)*d

printf("%d", N)
printf("%d", y)
```

C++ Implementation:-

```
#include <iostream>
#include <cmath>
using namespace std;
void quadraticMethod (double S, double a1,
                      double d) {
    double A = d;
    double B = 2*a1 - d;
    double C = -2*S;

    double discriminant = B*B - 4*A*C;

    if (discriminant < 0){
        cout<< "No real solution";
        return;
    }
    double N1 = (-B + sqrt(discriminant))
                / (2*A);
    double N2 = (-B - sqrt(discriminant))
                / (2*A);
    int N = 0
    if (N1>0 ){
        N = N1
    }else if
```

DATE: / /

```
        N = N2
    } else {
        cout << "No positive integer";
    }
    double  y = a1 + (N-1)*d;

    cout << "Quadratic Method: N= "<<N;
    cout << "Last term (y): << y;
}
```

Time Complexity :-

- Best Case : O(1)
- Average Case : O(1)
- Worst Case : O(1)

Algorithm performs basic arithmetic and square root calculations.

PROBLEM 2 :- Factorization Method

Algorithm :-

Algorithm Factorization (Sum, A1, Cd) {
[Algorithm to find N (no.of terms) and last term using factorization]
Step 1 : [Read the values]

Read (Sum, A1, Cd)

DATE: / /

Step 2: [Intialize the variables]

current_sum = 0

current_term = a1

N = 0

Step 3: [Perform loop to retrieve all terms and add]

Repeat step 4 while current_sum < S

Step 4: [Add values]

N = N + 1

current_sum = current_sum + current_term

If (current_sum == Sum) then

~~y = current_term~~

return

current_term = current_term + d

Step 5: [Calculate last term]

y = current_term

Step 6: [Output value]

Write (N, y)

Step 7: [Check the invalid scenario]

If (current_sum > S) then

return false

Step 8: [Finish]

Exit.

CamScanner

DATE: / /

Pseudocode:-

```
FactorizationMethod (S,a1,d) {
    current_sum = 0
    N = 0
    current_term = a1

    while (current_sum < S) {then
        y = current_term
        printf ("%d", N)
        printf ("%d", y)

    N = N+1
        current_sum = current_sum + current_term
        If (current_sum == S) {then
            y = current_term
            printf ("%d", N)
            printf ("%d", y)
            Return
        current_Term = current_Term + d
    End while

    Printf ("No valid sequence matches sum")
```

C++ Implementation:-

```
#include <iostream>
using namespace std;
```

DATE: / /

```
int main ( ){
    double a1, d, S;

    cout << "Enter A1:";
    cin >> a1;
    cout << "Enter d:";
    cin >> d;
    cout << "Enter Sum:";
    cin >> S

    int N = 1;
    double sum;

    while (True){
        sum = (N / 2.0)*(2 * a1 + (N-1)*d);
        If (sum == S)
            break;
        N++;
    }
    double y = a1 + (N+1)*d;
    cout << "Num of Terms:" << N << endl;
    cout << "last term:" << y
    return 0;
}
```

Time Complexity:-

Best case : O(1) [loop run exactly 1]

Average case : O(N)

Worst case : O(N) [Run exactly N times to reach Sum]

DATE: / /

Problem 3: Comparison of Methods

Feature	Quadratic	Factorization
(1) Time Complexity	$O(1)$ constant time	$O(N)$ linear time
(2) Efficiency	Highly efficient	Less efficient
(3) Space complexity	$O(1)$ constant memory usage	$O(1)$ constant memory usage.
(4) Practical usability	Ideal for standard mathematical applications, large datasets, or performance-critical systems	Good for educational purposes or scenarios where N is known to be very small. Vulnerable to performance bottlenecks on large sequences.

Problem 4: Generalization

Both algorithms already work for any input values of:

- First term a,
- Common difference d
- Sum S